

开源软件基础

开源软件仓库与典型软件仓库结构

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 12, 2019



- 开源软件许可证制度
 - GPL、BSD、MIT 等许可证协议
 - 许可证典型案例
- 开源软件生态系统
 - 全栈
 - 开源软件生态系统典型例子

Outline

① 开源软件仓库介绍

② 开源软件仓库结构

软件仓库

定义

A software repository is a storage location from which software packages may be retrieved and installed on a computer.

Many software publishers and other organizations maintain servers on the Internet for this purpose, either free of charge or for a subscription fee. Repositories may be solely for particular programs, such as CPAN for the Perl programming language, or for an entire operating system^a.

^ahttps://en.wikipedia.org/wiki/Software_repository

典型软件仓库

软件仓库分类

- 操作系统发行版：基于某种内核的操作系统及外围软件构成的完整系统，如 Debian、Fedora 等 GNU/Linux 发行版，或 FreeBSD、OpenBSD 等 BSD 发行版
- 特定语言仓库：提供针对某个语言的软件包/类库等，如 Perl 的 CPAN、R 的 CRAN、T_EX 的 CTAN、C++ 的 boost 等

软件仓库

软件仓库的操作

用户通过包管理器与软件仓库进行交互，功能包括搜索、安装、卸载、升级等功能。例如 Debian 及其衍生发行版的包管理器 APT (Advanced Packaging Tool)、Red Hat 及其衍生发行版的 yum、Arch Linux 的 Pacman 等等。

今天就能开始用的小知识

APT 梗

```
$apt moo
```

```

          (__)
          (oo)
        /-----\
       /  |       | \
      *  /\----/\
         ~~      ~~

```

```
... "Have you mooed today?" ...
```

```
$ LC_ALL=C apt |tail -n 1
```

```
$ apt |tail -n 1
```

This APT has Super Cow Powers.

本 APT 具有超级牛力。

软件仓库分类

按类型分

- 综合类：如 Apache、Mozilla 等基金会主导的仓库
- 针对特定语言：如 CRAN、CPAN、Pypi

按粒度分

- 微观：Pypi 中的某个包、Apache 下的子项目
- 宏观：GNU/Linux 发行版，如 Debian、Ubuntu

微观开源软件仓库

微观角度，针对某个特定功能的软件所对应的所有信息，组织在一起构成的软件仓库，一般包括的信息包括：

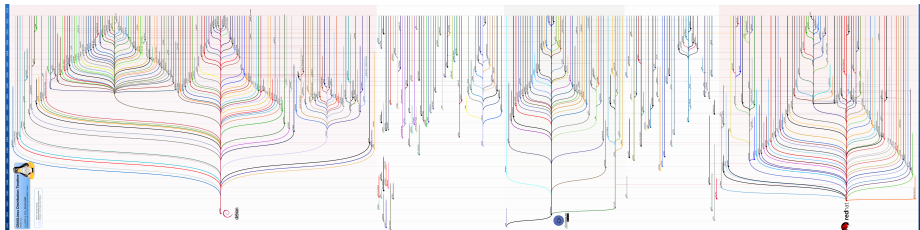
- 源代码
- 构建脚本，如 Makefile、build.xml 等
- 帮助文档
- 版本信息：如.git/，.hg/内用于版本控制的信息

以 Python 仓库为例

```
ren@home: /home/ren/play/temp/cpython/Python/ast.c
```

.git	clinic	4	/*
.github	_warnings.c	35.7 K	* This file includes functions to transform a concre
Doc	asdl.c	1.39 K	* an abstract syntax tree (AST). The main function i
Grammar	ast.c	161 K	*
Include	bltinmodule.c	81.3 K	*/
Lib	bootstrap_hash.c	18.3 K	#include "Python.h"
Mac	ceval.c	159 K	#include "Python-ast.h"
Misc	ceval_gil.h	9.04 K	#include "node.h"
Modules	codecs.c	44.9 K	#include "ast.h"
Objects	compile.c	155 K	#include "token.h"
Parser	condvar.h	8.86 K	#include "pythonrun.h"
PC	dtoa.c	78.5 K	
PCbuild	dup2.c	705 B	#include <assert.h>
Programs	dynamic_annotations.c	6.59 K	
Python	dynload_aix.c	5.7 K	static int validate_stmts(asdl_seq *);
Tools	dynload_dl.c	581 B	static int validate_exprs(asdl_seq *, expr_context_ty
.gitattrib~	dynload_hpux.c	1.88 K	static int validate_nonempty_seq(asdl_seq *, const ch
.gitignore	dynload_next.c	3.88 K	static int validate_stmt(stmt_ty);
.hgeol	dynload_shlib.c	3.31 K	static int validate_expr(expr_ty, expr_context_ty);
.hgignore	dynload_stub.c	186 B	
.hgtags	dynload_win.c	10 K	static int
.travis.yml	errors.c	31.9 K	validate_comprehension(asdl_seq *gens)
aclocal.m4	fileutils.c	42.1 K	{
config.gue~	formatter_unicode.c	50 K	int i;
config.sub	frozen.c	1.67 K	if (!asdl_seq_LEN(gens)) {
configure	frozenmain.c	2.7 K	PyErr_SetString(PyExc_ValueError, "comprehens

宏观开源软件仓库（Linux 发行版为例）



典型 Linux 发行版

Debian

Debian 是完全由自由软件组成的类 UNIX 操作系统，其包含的多数软件使用 GNU 通用公共许可协议授权，并由 Debian 计划的参与者组成团队对其进行打包、开发与维护。

Debian 项目最初由伊恩·默多克于 1993 年发起，Debian 0.01 版在 1993 年 9 月 15 日发布，而其第一个稳定版本则在 1996 年发布。^a

^a<https://zh.wikipedia.org/zh-cn/Debian>



典型 Linux 发行版

Debian 的特色

Debian 以其坚守 Unix 和自由软件的精神，以及其给予用户的众多选择而闻名。现时 Debian 提供了超过 25,000 个软件，超过 50,000 个软件包，并正式支持 10 个计算机系统结构。

作为一个大的系统组织框架，Debian 旗下有多种不同操作系统核心的分支计划，主要为采用 Linux 核心的 Debian GNU/Linux 系统，其他还有采用 GNU Hurd 核心的 Debian GNU/Hurd 系统、采用 FreeBSD 核心的 Debian GNU/kFreeBSD 系统等。众多知名的 Linux 发行版，例如 Ubuntu、Knoppix 和 Deepin，也都建基于 Debian GNU/Linux。

典型 Linux 发行版

今天就能开始用的小知识

- Debian 名字来源于发起人 Ian Murdock 和当时的女友 Debra Lynn
- Debian 的版本号昵称来自玩具总动员
- Debian 的 logo 来自玩具总动员角色巴斯光年的胡子（存疑）



Debian



Buzz 1.1
1996



Rex 1.2
1996



Bo 1.3
1997



Hamm 2.0
1998



Slink 2.1
1999



Potato 2.2
2000



Woody 3.0
2002



Sarge 3.1
2005



Etch 4.0
2007



Lenny 5.0
2009



Squeeze 6.0
2011



Wheezy 7.0
2013



Jessie 8.0
2015

Stretch 9.0
2017?



Buster 10.0
2019?

典型 Linux 发行版

Ubuntu

- Ubuntu 是以桌面应用为主的 Linux 发行版，Ubuntu 由 Canonical 公司发布，他们提供商业支持。它是基于自由软件，其名称来自非洲南部祖鲁语或科萨语的 ubuntu 一词，意思是“人性”、“我的存在是因为大家的存在”，是非洲传统的一种价值观。
- Ubuntu 在 Ubuntu 12.04 的发布页面上使用了“友帮拓”作为官方译名。



Ubuntu

今天就能开始使用的小知识

Ubuntu 的发行代码规律

4.10	Warty Warthog	多疣的疣猪
5.04	Hoary Hedgehog	白发的刺猬
5.10	Breezy Badger	活泼的獾
6.06LTS	Dapper Drake	整洁的公鸭
6.10	Edgy Eft	尖利的小蜥蜴
7.04	Feisty Fawn	烦躁不安的鹿
7.10	Gutsy Gibbon	胆大的长臂猿
8.04LTS	Hardy Heron	坚强的鹭
8.10	Intrepid Ibex	无畏的羴羊
9.04	Jaunty Jackalope	活泼的鹿角兔
9.10	Karmic Koala	幸运的茶袋熊
10.04LTS	Lucid Lynx	清醒的山猫

Ubuntu

今天就能开始使用的小知识

Ubuntu 的发行代码规律

10.10	Maverick Meerkat	标新立异的狐獴
11.04	Natty Narwhal	敏捷的独角鲸
11.10	Oneiric Ocelot	有梦的虎猫
12.04LTS	Precise Pangolin	精准的穿山甲
12.10	Quantal Quetzal	量子的格查尔鸟
13.04	Raring Ringtail	铆足了劲的环尾猫熊
13.10	Saucy Salamander	活泼的蝾螈
14.04LTS	Trusty Tahr	可靠的塔尔羊
14.10	Utopic Unicorn	乌托邦的独角兽
15.04	Vivid Vervet	活泼的长尾黑颞猴
15.10	Wily Werewolf	老谋深算的狼人
16.04LTS	Xenial Xerus	好客的非洲地松鼠
16.10	Yakkety Yak	喋喋不休的牦牛
17.04	Zesty Zapus	热情的美洲林跳鼠
17.10	Artful Aardvark	巧妙的土豚
18.04LTS	Bionic Beaver	仿生的海狸

典型 Linux 发行版

ArchLinux

Arch Linux 是朝向轻量以及简单的 Linux 发行版。其中“简单”被定义为“避免不必要或复杂的修改”，也就是说，是由开发者角度定义，而非用户角度思考。^a

- 简洁：避免任何不必要的添加、修改和复杂增加。
- 现代：尽全力保持软件处于最新的稳定版本，采用滚动升级策略，安装之后可以持续升级。
- 实用：开发者依赖基于事实的技术分析和讨论，避免政治因素，不会被流行观点左右。仓库中既提供了开源、自由的软件，也提供了闭源软件。实用性大于意识形态。
- 以用户为中心：Arch Linux 适用于乐于自己动手的用户，他们愿意花时间阅读文档，解决自己的问题。

^ahttps://zh.wikipedia.org/zh-cn/Arch_Linux



典型 Linux 发行版

Red Hat

Red Hat Linux 是由 Red Hat 公司发行的一个 Linux 发行包。其 1.0 版本于 1994 年 11 月 3 日发行。虽然其历史不及 Slackware 般悠久，但比起很多的 Linux 发行包，Red Hat 的历史悠久得多。Red Hat Linux 中的 RPM 软件包格式可以说是 Linux 社区的一个事实标准，被广泛使用于其他 Linux 发行包中。



典型 Linux 发行版

Fedora

Fedora Linux 是较具知名度的 Linux 发行包之一，由 Fedora 项目社区开发、红帽公司赞助，目标是创建一套新颖、多功能并且自由（开放源代码）的操作系统。

Fedora 对于用户而言，是一套功能完备、更新快速的免费操作系统；而对赞助者 Red Hat 公司而言，它是许多新技术的测试平台，被认为可用的技术最终会加入到 Red Hat Enterprise Linux 中。^a

^a<https://zh.wikipedia.org/zh-cn/Fedora>



典型 Linux 发行版

Gentoo

Gentoo Linux 是一种 Linux 操作系统，基于 Portage 包管理系统，而拥有几乎无限制的适应性特性，被官方称作元发行版（meta-distribution），支持多达 10 种以上的电脑系统结构平台。此项目和它的产品以巴布亚企鹅命名。Gentoo 包管理系统的设计是模块化、可移植、易维护、灵活以及针对用户机器优化的。

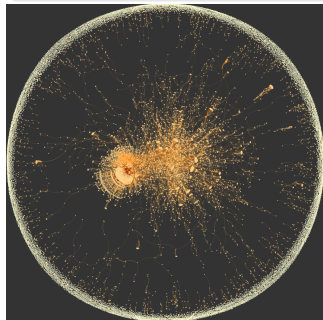


特定语言仓库

Pypi

PyPI(Python Package Index) 是 python 官方的第三方库的仓库，所有人都可以下载第三方库或上传自己开发的库到 PyPI。PyPI 推荐使用 pip 包管理器来下载第三方库。^a

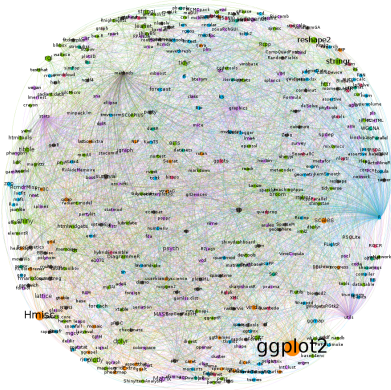
^a<http://kgullikson88.github.io/blog/pypi-analysis.html>



特定语言仓库

CRAN

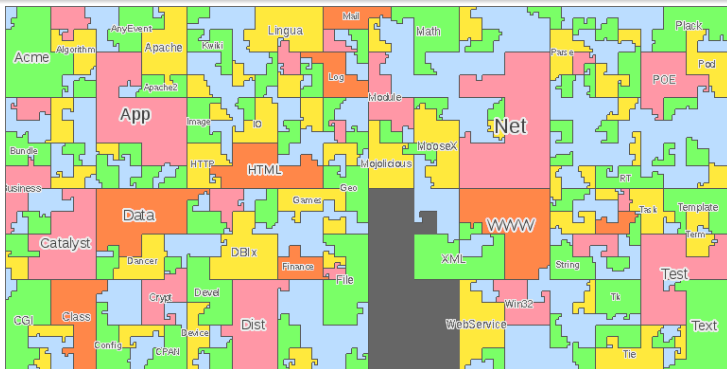
NASAC 2017 Keynote: Knowledge Flows in Open Source Software Supply Chains – Audris Mockus



特定语言仓库

CPAN

<http://mapofcpan.org/>



Outline

① 开源软件仓库介绍

② 开源软件仓库结构

以 Debian 为例

Debian 的基本结构

涵盖的内容

- Debian 软件类型
- Debian 仓库结构
- Debian 发布周期
- Debian 发布流程
- Debian 依赖管理
- Debian 缺陷管理

Debian 软件类型

Debian 社群契约^a

^ahttps://www.debian.org/social_contract.zh-cn.html

- 1 Debian 将始终是 100% 的自由软件
- 2 我们将回报自由软件社群
- 3 我们绝不隐瞒问题
- 4 我们将优先考虑我们的用户及自由软件
- 5 哪些作品不符合我们的自由软件规范

Debian 软件类型

Debian 将始终是 100% 的自由软件

我们制订一个名为"Debian 自由软件指导方针"的标准便于我们判定某项作品"自由"与否。我们保证 Debian 系统机器附带的软件包遵循这些自由软件的方针。我们将同时支持在 Debian 上开发及使用自由或者非自由软件的用户。但是，我们决不让这个系统依赖于任何非自由软件。

Debian 软件类型

我们将回报自由软件社群

当我们编写 Debian 系统的新的部件之时，我们将使其遵循 Debian 自由软件指导方针的理念。我们将尽最大努力，打造最优秀的系统，以利自由软件得到最广泛的使用及传播。我们将反馈那些作品被我们系统收录的"上游"作者，例如缺陷的修正、改良的意见以及用户的需求等这些信息。

Debian 软件类型

我们绝不隐瞒问题

我们将始终把我们整个的缺陷报告数据库开放给公众阅读。由用户在线提交的报告，将会很快的出现在其他人的眼前。

Debian 软件类型

我们将优先考虑我们的用户及自由软件

我们由我们的用户及自由软件社群的需要所导向。我们将优先考虑他们的利益。我们将在多种计算环境中支持我们的用户的操作需要。我们不反对在 Debian 系统上使用非自由软件，我们也不会尝试向创建和使用这部分软件的用户索取费用。我们允许他人，在没有我们的资金的参与下，制造包括 Debian 以及商业软件的增值套件。为了达成这些目标，我们将提供一集成的、高质量的、100% 自由的软件，而不附加任何可能阻止在这些方面使用的法律限制。

Debian 软件类型

哪些作品不符合我们的自由软件规范

我们明了，某些我们的用户需要使用不符合 Debian 自由软件指导方针的作品。我们为这些作品，在我们的 FTP 库中留出了"contrib" 以及"non-free" 目录。在这些目录下的软件包，并不属于 Debian 系统尽管它们已被配置成可以在 Debian 下使用。我们鼓励光盘制造商阅读这些目录下的软件的许可证，以判断他们是否可以在光盘中发行这些软件。所以，尽管非自由软件并非 Debian 系统的一部分，我们仍支持它们的使用，并且我们为非自由软件提供了公共资源 (诸如我们的缺陷跟踪系统以及邮件列表)。

Debian 软件类型

Debian 自由软件指导方针 (DFSG)

- 自由的再次发行

Debian 组件的许可证不得限制任何一方将此软件作为含有若干不同来源的程序的一套软件集合中的一个组件用于销售或者捐赠。该许可证不得向诸如此类销售行为的销售方索取专利费或者其它费用。

- 源代码

程序必须包括源代码，而且必须允许以源代码以及预先编译好的形式发行。

- 作品的衍生

许可证必须允许对其所属作品的修改以及衍生，而且必须允许这些作品在原始软件的许可证条款下发行。

Debian 软件类型

Debian 自由软件指导方针 (DFSG)

- 作者源代码的完整性

许可证只有在允许"补丁文件"随其所属作品的源代码一同发行,以便在编译时修改程序的情况之下,方可限制对其所属作品的源代码在发行时的修改行为。许可证必须清楚表明用已修改的源代码编译而成的软件,是允许发行的。许可证可要求衍生软件使用有别于原来软件的名称或者版本号。(这是一种妥协,Debian 组织鼓励所有作者不要限制任何源代码文件或者二进制文件的修改。)

- 禁止歧视人士或者组织

许可证不能歧视任何人士或者由多人组成的组织。

- 禁止歧视用途

许可证不能歧视程序可以被用于的任何特定领域。例如,许可证不得限制程序用于商业或者基因研究。

Debian 软件类型

Debian 自由软件指导方针 (DFSG)

- 许可证的发行
程序附带的权利必须适用于程序再次发行的每一个受众，无需他们再执行一个附加的许可证。
- 许可证不能特定于 Debian
程序附带的权利不能由该程序是否为 Debian 的一部分来决定。如果这个程序从 Debian 中摘取出来，即使在 Debian 之外但仍然在该程序的许可证条款下使用或者发行，那么它再次发行的每一个受众都将拥有和那些在该程序与 Debian 系统结合时被授予的完全相同的权利。
- 许可证的规定不得污染其他软件
许可证不得对其他与此软件一同分发的软件作出任何限制的规定。例如，许可证不得要求所有与它在同一媒体中一同分发的软件都是自由软件。

Debian 软件类型

Debian 自由软件指导方针 (DFSG)

- 许可证示例

"GPL"、"BSD" 和 "Artistic" 均是我们视为"自由"的许可证的示例。我们"与自由软件社群订立"社群契约"的这个概念是由 Ean Schuessler 提出的。本文是由 Bruce Perens 写成初稿,并由其他 Debian 开发者在 1997 年 6 月起长达一个月的电子邮件会议中精炼而成。后来被收录作为 Debian 项目所公布的政策。

此后, Bruce Perens 从 Debian 自由软件指导方针中删除特别提到 Debian 的部份,制成了"开放源码定义"。

欢迎其他组织以这份文档为基础进行衍生和扩写。若您这样做,请注明原文出处"Debian 计划",以表达对我们工作的赞赏和肯定。

Debian 仓库结构

mirror.dlut.edu.cn/debian/dists/stable/

Index of /debian/dists/stable/

Name	Last Modified	Size	Type
Parent Directory/		-	Directory
contrib/	2017-Jul-22 17:03:40	-	Directory
main/	2017-Jul-22 17:03:40	-	Directory
non-free/	2017-Jul-22 17:03:40	-	Directory
ChangeLog	2017-Jul-22 16:01:11	34.2K	application/octet-stream
Release	2017-Jul-22 17:03:40	115.1K	application/octet-stream
Release.gpg	2017-Jul-22 17:26:53	2.3K	application/octet-stream

lighttpd/1.4.28

Debian 发布周期

Debian 的发布周期

Debian 主要分三个版本：稳定版本（stable）、测试版本（testing）、不稳定版本（unstable）。目前的稳定版本为 Debian Buster，上一个稳定版本是 Stretch，不稳定版本永远为 Debian sid。

Debian 稳定版通常每隔两年发布一个版本，自发行后会得到为期约三年的正式支持，期间会不定期得到小版本更新与持续的安全更新以修复发现的重要问题。

自 Debian 6 开始，Debian 开始了长期支持计划，在每个稳定版三年支持期结束后由长期支持团队提供额外的两年安全更新支持，但不会发布小版本。故目前的稳定版可以得到总计五年的安全更新支持。

Debian 发布周期



Buster

Debian 发布周期



Bullseye

Debian 发布周期



Sid



Debian 发布流程

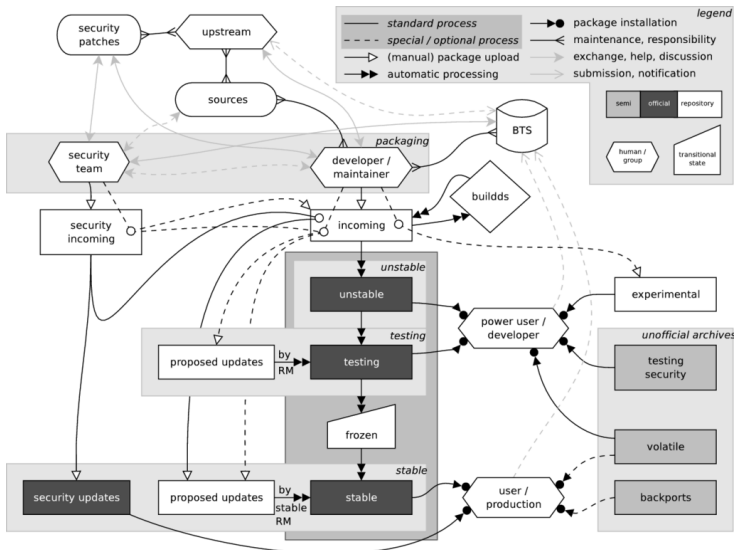
Debian 的发布流程

正在开发中的软件会被上传到名为“不稳定”（unstable，代号 sid）和“实验性”（experimental）的项目分支上。上传至“不稳定”分支上的软件通常是由软件的原开发者发布的稳定版本，但包含了一些未经测试的 Debian 内部的修改（例如软件的打包）。而未达到“不稳定”分支要求的软件会被置于“实验性”分支。

一套软件在置于“不稳定”分支一段时间后（关乎软件修改的紧急性），该软件会自动被移至“测试”分支。但如果软件有严重错误被报告，或其所依存的软件未合乎“测试”分支的要求，该软件则不会被移至“测试”分支。

当在“测试”分支中的软件 Bug 总数低于一特定数量后，“测试”分支会成为下一个稳定版本。

Debian 发布流程



Debian 依赖管理

Debian 的依赖管理

```
$dgbtree nano | dot -Tpng > nano.png
digraph "nano" {
    rankdir=LR;
    node [shape=box];
    "nano" -> "libncursesw5" [color=blue,label="(>= 6)"];
    "libncursesw5" -> "libtinfo5" [color=blue,label="(= 6.0+20170902-1)"];
    "libncursesw5" -> "libgpm2";
    "nano" -> "libtinfo5" [color=blue,label="(>= 6)"];
    "nano" -> "pico" [color=red];
    "nano" [style="setlinewidth(2)"]
    "pico" [style=filled,fillcolor=oldlace];
}
// Excluded dependencies:
// libc6 zlib1g
// total size of all shown packages: 3093504
// download size of all shown packages: 976236
```

Debian 依赖管理

Package: nano

Priority: standard

Section: editors

Installed-Size: 716

Architecture: amd64

Version: 2.6.3-1

Replaces: pico

Provides: editor

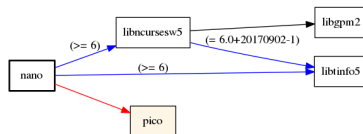
Depends: libc6 (>= 2.14), libncursesw5 (>= 6), libtinfo5 (>= 6)

Suggests: spell

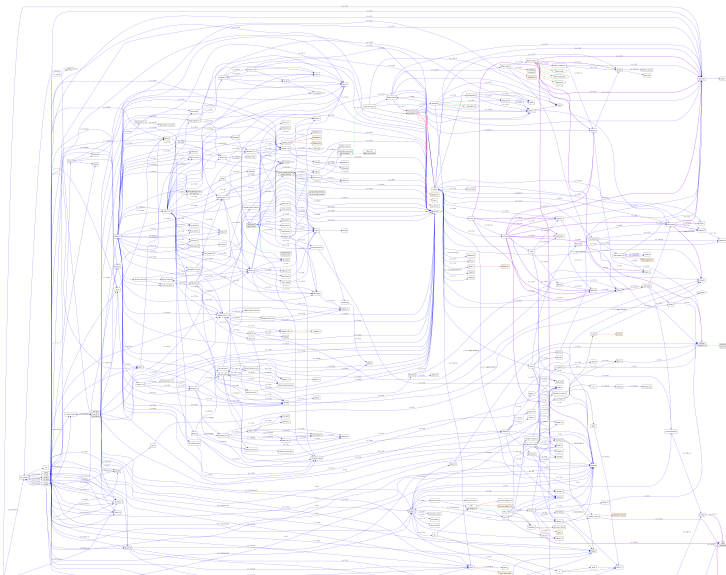
Conflicts: pico

Description-en: small, friendly text editor inspired by Pico

GNU nano is an easy-to-use text editor originally designed as a replacement for Pico, the ncurses-based editor from the non-free mailer package Pine (itself now available under the Apache License as Alpine).

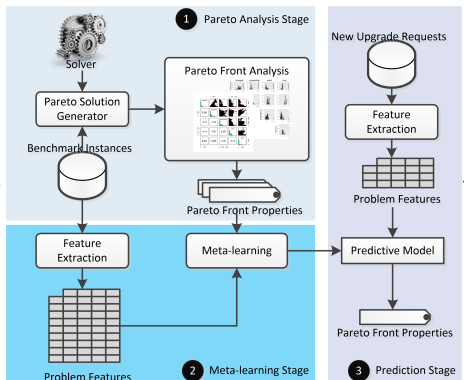


Debian 依赖管理



Debian 依赖管理

Debian 软件包的安装、升级和卸载可以建模为经典的可满足问题 (Satisfiability), 并且是多目标问题, 如最小化安装文件数、最小化无法升级软件数、最小化新软件包个数等等。这些目标之间可能存在冲突。

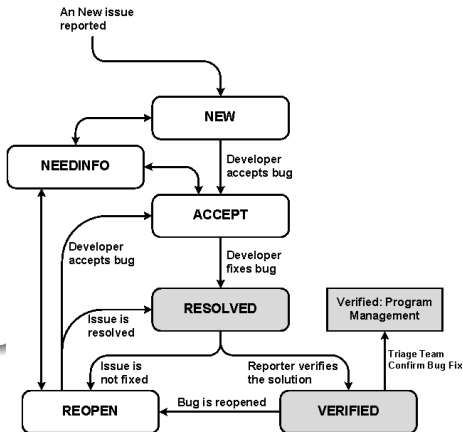


¹ Zhilei Ren, He Jiang, Jifeng Xuan, Yan Hu. Analyzing Inter-objective Relationships: A Case Study of Software Upgradability. Proceedings of PPSN 2016, Edinburgh, Scotland, UK. September 17-21, 2016, pp. 442-452. [📄](#) [🔍](#) [🔗](#) [🔄](#) [📌](#)

Debian 缺陷管理

Debian 的 Bug 管理

Debian 的缺陷使用 BTS 系统 (Bug Tracking System) 进行管理。一个新的缺陷在报告后会经历 NEW、ACCEPT、RESOLVED 等流转状态，直至最终修复并且验证。

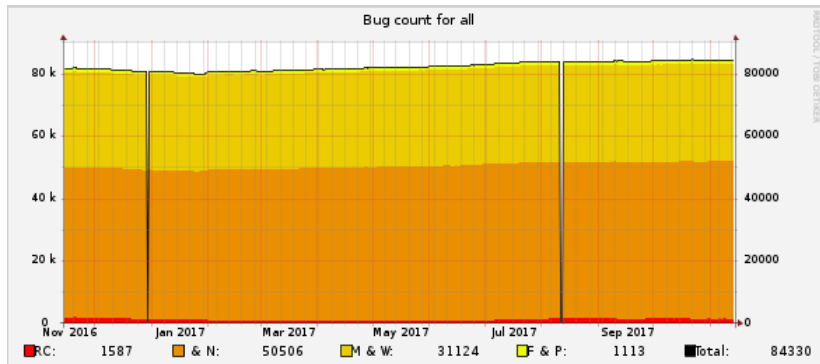


Debian 缺陷管理

Debian 缺陷严重级别

- critical: 使其他不相关软件包不可用, 或造成数据丢失、安全漏洞等问题
- grave: 使问题软件包本身不可用或造成数据丢失、安全漏洞等
- serious: 与 Debian 政策违背, 或者使软件包不适合发布
- important: 影响软件可用性, 尚不至于完全不可用
- normal: 默认选项, 大多数属于此类
- minor: 不影响可用性, 并且修复相对较容易
- wishlist: 特性请求 (Feature Requests), 以及由于软件设计原因导致难于修复

Debian 缺陷管理



<https://www.debian.org/Bugs/>